SENTINEL AI — Toolchain & Competition Compliance

Which tools we use, why, and how each competition requirement is satisfied

Mohammed Ilyas · first issued 20 Sep 2026 · updated 24 Sep 2026 (post-review) v1.2

> Third of three documents. The Master Document covers what and why; the Technical Document covers how it is built; this one covers what we build it *with*.

---

1. Stack decision

MEAN on Google Cloud. Angular 20, Node.js on Cloud Run, MongoDB Atlas hosted on Google Cloud, Gemini through Vertex AI.

Layer	Choice	Why this one
Frontend	Angular 20, standalone components	It is the team's strongest skill. Eight years of enterprise Angular is a competitive advantage we should spend, not set aside
Backend	Node.js 22, Express, TypeScript	Shares the `domain` and `rules` packages with the frontend verbatim — one implementation of the rule engine, running in both places
Database	MongoDB Atlas, deployed into Google Cloud	Document shape matches `EvidenceBundle` and `RiskAssessment` with no ORM mapping; change streams give us a push feed without a second message bus
Runtime	Cloud Run	Named in the competition FAQ as an accepted deployment target
Messaging	Pub/Sub	Ordering keys per account, which the rapid-drain arithmetic requires
AI	Gemini on Vertex AI	Mandatory in substance: the solution must make meaningful use of Google AI

What changed from the prototype

The prototype is one self-contained HTML file with the rule engine inline. Nothing about its **behaviour** changes — the seven rules, the thresholds, the 95-point ladder and the policy gate move across unmodified. What changes is where they live:

Prototype	Project
Rules inline in one `<script>`	`packages/rules` — pure functions, no I/O, unit-tested, imported by both apps
Scenario in a JS array	`packages/scenarios` — seeded generator, same `DEMO_TAKEOVER_01` output
State in a `S` object	NgRx for domain state, Signals for UI state
No persistence	MongoDB Atlas collections
Simulated Gemini output	Vertex AI calls with response schemas
Simulated stream	Pub/Sub → Cloud Run → WebSocket

The one decision that is not negotiable

`packages/rules` has no framework dependency and no I/O. It runs identically in Node and in the browser Web Worker. That is what lets the client compute an optimistic score without a second implementation drifting from the first — and it is what keeps `contributions[]` the single source of truth when the same numbers appear in the UI, in the database, in the evidence bundle and in the case report.

---

2. Competition requirements

Taken from the official FAQ and themes page.

Requirement	What satisfies it
Built on Google Cloud's tech stack; other clouds are ineligible for evaluation	Cloud Run, Pub/Sub, Vertex AI, Artifact Registry, Cloud Build, Secret Manager, Cloud Logging — all on one GCP project
Deployed on Cloud Run, GCP or Firebase	Four Cloud Run services (`core` , `stream` , `ui` , `simulator`)
Meaningful use of Google AI	Gemini on Vertex AI: explanation, evidence correlation, case summary, report draft — with structured output schemas
Working prototype, full product not required	The eight MVP modules in the Master Document
Deployed working link	Public Cloud Run URL for `sentinel-ui`
Video demo under 3 minutes	The ten-scene narrative, already time-budgeted to 3:00
Public GitHub repository	`sentinel-ai` monorepo, README that runs from a clean clone
Detailed deck	Built from the Master Document
One submission, one theme; all materials in English	BFSI — Intelligent Risk, Fraud & Financial Experiences
Team of 2–4, all JAPAC-based, 2 travel to Singapore	To confirm at team formation

Timeline — confirmed on the official site

Phase	Dates
Registration & team formation	1 September – 11 October 2026
Prototype building & submission	7 September – 18 October 2026
Submission deadline	18 October 2026
Evaluation	19 October – 6 November 2026
Finalists announced	7 November 2026
Grand Finale, Singapore	4 December 2026

The earlier confusion about a 4 October registration close is resolved: registration runs to **11 October**, and the build window opened on **7 September**.

The eligibility question, answered

The FAQ says **"Only fresh projects built during the hackathon timeline are eligible."** The timeline for building is 7 September – 18 October. Everything on SENTINEL — these documents, the rule engine, the scenario, the prototype — began on 20 September, **inside that window**. Nothing pre-dates it.

So the rule is satisfied on the facts, and the discipline that follows is simply good practice rather than damage control:

- The submitted repository is created fresh with its own first commit, and every commit is inside the window.
- No code is imported from any earlier project.
- The HTML prototype is described honestly in the deck as a design prototype that preceded the production build.

Still worth one email to the organisers confirming that design work and planning inside the window are not a problem — but this is now a confirmation, not a risk.

Language

The themes page is explicit: **"All submission materials, including code, documentation, and presentations, must be in English."**

That draws a clean line for the team:

Internal	Submitted
Tamil NotebookLM audio, Q&A, architecture walkthroughs, rehearsal	Proposal PDF
Any language that gets the team to real understanding fastest	README and all code comments
Whiteboard notes, standups	Deck, 3-minute video and its captions

Tamil material for team comprehension is a good idea and costs the submission nothing — provided none of it is what gets uploaded.

Submission deliverables, exactly as stated

1. **Clear proposal** — a PDF presentation covering problem, solution benefits, impact and feasibility
2. **Functional prototype** — built with Google Cloud AI tools, deployed on Cloud Run, GCP or Firebase
3. **Comprehensive documentation** — a public video, 3 minutes maximum, on YouTube, Vimeo or Google Drive
4. **Category specification** — the chosen theme, named explicitly

---

3. Google Cloud services

One GCP project, `sentinel-ai-prod`, in `asia-southeast1` (Singapore) — the region matches the regulatory story and keeps latency low for the finale.

Service	What it carries	Notes
Cloud Run	`sentinel-core`, `sentinel-stream`, `sentinel-ui`, `sentinel-simulator`	`min-instances=1` on the three services (core, stream, ui). The simulator is a Cloud Run Job, run on demand; jobs have no minimum instances. Cold starts are the most common way a hackathon demo dies in its first fifteen seconds

Service	What it carries	Notes
Pub/Sub	Transaction event transport	Ordering keys set to `accountId`; rapid-drain arithmetic is order-dependent
Vertex AI	Gemini calls	Called with the service account's own credentials, no API keys
Artifact Registry	Container images	One repository, four images
Cloud Build	CI on push to `main`	Typecheck, unit tests, scenario assertion, build, deploy on green
Secret Manager	Atlas connection string	The only secret in the system
Cloud Logging / Monitoring	Structured logs, one dashboard, one budget alert	
Cloud DNS + a domain	Optional	Only if a memorable URL is worth the hour

What we deliberately do not use

Not using	Why
Firestore	MongoDB Atlas is the chosen store. Running both would mean two models of the same data
BigQuery	The 24-hour rolling window and the two-hop graph are computed in Mongo. Revisit only if aggregation performance actually fails
GKE	Four containers do not need an orchestrator. It would read as enterprise theatre
Apigee, Cloud Armor, Identity Platform	Production concerns we are honest about not solving in a prototype
Vertex AI training, AutoML, model tuning	The risk engine is deterministic by design. Adding a trained model would weaken the architecture, not strengthen it

That last row is worth saying out loud in the pitch. The obvious move in an AI hackathon is to train something. Not training anything, and being able to explain precisely why, is a stronger technical position than a model nobody can interrogate.

---

4. Google AI tooling

Models

Per the I/O 2026 developer highlights, the current generation is **Gemini 3.5 Flash** — described as the core model for agents and roughly four times faster than other frontier models — with **Gemini 3.1 Pro** as the prior baseline.

Tier	Model	Jobs	Budget
Fast	Gemini 3.5 Flash	Risk explanation, alert enrichment, entity summary, next-step suggestion	First token under 800 ms, streamed

Tier	Model	Jobs	Budget
Deep	Gemini 3.1 Pro	Case summary, multi-source evidence correlation, case report draft	Complete under 8 s

Model identifiers live in **Cloud Run environment config**, resolved through a single `ModelRouter.resolve(job)`. Never a literal in code. Re-verify the available IDs on **5 October** and again the week before the finale — a retired model ID on stage is avoidable and fatal.

How Gemini is called

Always through **Vertex AI**, from `sentinel-core`, with the service account's own credentials. Never from the browser, never with an API key, never with a prompt built by string concatenation. The input is one typed `EvidenceBundle`; the output is validated against a response schema before it reaches the UI.

The constraint that is the product

The system instruction is quoted verbatim in the submission:

- You are explaining a decision already made by a deterministic rule engine
- You may not add, remove or re-weight any risk contribution
- You may not assert a fact absent from the evidence bundle
- You may not recommend releasing a held transaction
- Where evidence is insufficient, say so rather than inferring

Every claim carries a `sourceField`, and the backend validates that the named field actually exists in the bundle it sent. A claim citing nothing is dropped before it reaches the analyst. This is cheap to build and it is the single most convincing thing to show a BFSI judge.

Where Google AI Studio fits

Prompt development, not runtime. Iterate the system instruction and response schemas in AI Studio, then move the settled prompt into the repo as a versioned file that `sentinel-core` loads. The application never calls AI Studio; it calls Vertex AI. Keeping that boundary clean means the prompt is reviewable, diffable and reproducible — which matters when a judge asks how the explanation is controlled.

---

5. Development environment

Firebase Studio is not an option

Google is sunsetting it. New workspaces and signups were disabled on **22 June 2026**, and the environment shuts down on **22 March 2027** (migration guide). Core Firebase services are unaffected — only the development environment is going. Do not start a four-week build in a product with a shutdown date.

Google's stated successors are **Google AI Studio** for browser-based prototyping and **Google Antigravity** for code-first agentic development.

What we actually use

Tool	Used for	Who
Google Antigravity	Agent-first development: desktop app, CLI and SDK, with direct connection to Google Cloud projects	Whoever prefers agent-driven work; optional per person
Google AI Studio	Prompt and schema iteration for the Gemini calls	The AI owner
Gemini Code Assist	In-editor assistance	Optional per person
VS Code + Angular CLI + Node 22 locally	The default. Fast, offline-capable, and nobody loses a day learning a new environment in week one	Everyone
Cloud Shell	`gcloud` work, quota checks, one-off deploys	Backend owner

The position on tooling: these are productivity choices, and none of them is a scoring criterion. Judges score the deployed artefact, the repository and the demo. A team member who is fastest in VS Code should use VS Code. The one thing that is not optional is that everything ships from `main` through Cloud Build to Cloud Run.

Worth saying in the submission

That we used Google's own agent tooling to build an application about constraining agents is a genuinely good line, and it is true rather than decorative — provided we actually use it. Name it in the deck only if we do.

---

6. Data layer — MongoDB Atlas

Atlas M0 or M10, deployed into Google Cloud, `asia-southeast1`. Same region as Cloud Run, so the hop is short and the data-residency story stays clean.

Why Mongo genuinely suits this workload

Not "because we chose it" — there are three real reasons:

1. **The documents are already documents.** `EvidenceBundle` nests an account profile, an assessment with a contributions array, a drain state, transaction and device history, and a graph neighbourhood. In a relational store that is eight joins to assemble the one object Gemini receives. In Mongo it is one read.
2. **Change streams replace a second message bus.** `sentinel-stream` tails the `interventions` and `alerts` collections and pushes to the WebSocket. No polling, no extra infrastructure.
3. **The 24-hour rolling window is one aggregation.** `$match` on account and timestamp, `$group` to sum outflow, `$merge` into the cached window on the account document. It is one pipeline against the same store the control path already reads, so no second analytical system is needed.

Collections

Collection	Holds	Key indexes
`accounts`	Hot state, cached drain window, known devices and payees	`_id`; `{institutionId, protected}`
`transactions`	Full history	`{sourceAccountId: 1, ts: -1}`; TTL on demo data

Collection	Holds	Key indexes
`assessments`	`RiskAssessment` including `contributions[]`	`{transactionId: 1}`; `{accountId: 1, ts: -1}`
`interventions`	Append-only audit log	`{accountId: 1, actedAt: -1}`; change stream watched
`alerts`	Open alert queue	`{status: 1, score: -1}`; change stream watched
`cases`	Investigation state and Gemini output	`{accountId: 1}`
`entities` / `entity_edges`	Graph nodes and edges	`{from: 1}`, `{to: 1}` for two-hop traversal

Rules that hold in the schema

- **Money is integer minor units.** SGD cents, never a float, never `Decimal128` unless a real need appears. `PROTECTED_BALANCE` is `5_000_000`.
- **Timestamps are UTC ISO 8601**, localised only at render.
- **`interventions` is append-only.** A release does not mutate the hold; it inserts a new record. The audit trail is a product feature, not a log.
- **`contributions[]` is written once by the engine and never updated.** No code path anywhere may modify a stored contribution — that is the database-level expression of "Gemini cannot re-weight the score".

Connection

One connection string, in Secret Manager, injected into `sentinel-core` and `sentinel-stream` as an environment variable. Atlas network access restricted to the Cloud Run egress range. The Mongo driver's connection pool is created once at module load, never per request.

---

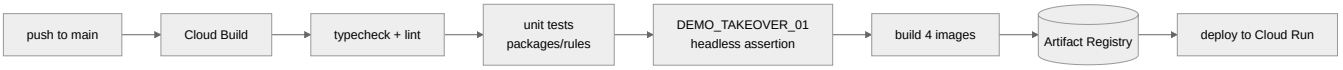
7. Build and deployment

Repository

`sentinel-ai`, public on GitHub, npm workspaces, single `package.json` at root.

```
sentinel-ai/
 apps/
   ui/          Angular 20 SPA           → Cloud Run (static container)
   core/        Express + risk + Gemini  → Cloud Run
   stream/      WebSocket gateway        → Cloud Run
   simulator/   Transaction generator    → Cloud Run job
 packages/
   domain/      types, constants, thresholds
   rules/       the 7 rules + tests, pure, no I/O
   scenarios/   seeded generators, DEMO_TAKEOVER_01
 infra/         cloudbuild.yaml, Dockerfiles, deploy scripts
 docs/          architecture diagram, submission material
```

Pipeline



The test that matters

One headless run of `DEMO_TAKEOVER_01` asserting:

- final score is exactly **95**
- `contributions[]` sums to the displayed score
- `thresholdMet` becomes true on the fourth transfer
- intervention state is `HELD`
- the six contribution codes and point values match the documented weights

If that fails, the demo is broken and we know before the judges do. It is the same mechanism as the seeded scenario — determinism is what makes both the demo safe and the test possible.

Deploy discipline

Everything ships from `main`. No manual `gcloud run deploy` from a laptop after 12 October — an undeployable `main` on submission day is a self-inflicted wound. Tag the submitted commit `submission-v1` and record the digest in the README.

---

8. Security and observability

Security posture, stated honestly

This is a prototype, and the submission says so. The things that would embarrass us are cheap to get right:

Control	How
No secrets in the repository	Atlas connection string in Secret Manager; `.env.example` with placeholders only
No API keys for Vertex AI	Service account credentials; Vertex is never called from the browser
Least privilege	`sentinel-core` gets Vertex AI User and Pub/Sub Subscriber, nothing more
Database exposure	Atlas network access limited to the Cloud Run egress range
No real personal data	Stated in the README's first paragraph; synthetic identifiers that do not resemble real NRICs, card or account numbers
CORS	Restricted to the deployed UI origin
Demo authentication	A single labelled demo login, never presented as an auth system
Analyst identity	Every intervention records an actor; the audit trail is a feature

Not claimed: production authentication, authorisation, encryption at rest beyond platform defaults, or penetration testing. Claiming those invites a question we cannot answer well. *"A functional prototype of a control, with the security posture of a prototype"* costs nothing and is unassailable.

Observability

What	Where
Structured JSON logs with `accountId`, `transactionId`, `ruleCode`	Cloud Logging
One dashboard: control-path latency, Gemini latency, Pub/Sub lag, error rate	Cloud Monitoring
Budget alert	Set in week one, at a figure the team agrees
In-app performance panel	Behind a key combination, shows live event rate and main-thread time — useful when a judge asks whether it really runs in real time

The one metric to instrument first

Control-path latency: Pub/Sub publish → rendered, scored, gated row. Target under 500 ms. That is the number we quote on stage, so it should be measured rather than asserted.

---

9. Cost

Four weeks of build plus the finale. The point of this section is that cost is checked weekly rather than discovered on 17 October.

Item	Driver	Control
Cloud Run, 3 services at `min-instances=1`	The largest single line	1 vCPU / 512 MiB each; do not raise min-instances
Vertex AI	Tokens per call	Evidence bundle caps (50 transactions, 40 graph nodes) and explanation caching by `assessmentId`
MongoDB Atlas	Cluster tier	M0 free tier is likely sufficient; M10 only if change streams or connection limits force it
Pub/Sub	Message volume	Negligible at demo rates
Artifact Registry, Cloud Build, Logging	Storage and build minutes	Negligible

Actions in week one, not later

1. **Check whether the competition provides Google Cloud credits.** Many Google hackathons do. If credits exist, claim them before provisioning anything — this is the single highest-value hour of setup work.
2. Set a **billing budget alert** at an agreed figure with email to all team members.
3. **Request Vertex AI quota** on the project. Discovering a denied quota on 14 October is the failure mode; discovering it on 23 September costs nothing.
4. Confirm the Atlas free-tier limits against our expected document counts.

The honest position

If no credits are available, this stack runs for four weeks on a small personal spend, dominated by keeping three Cloud Run service instances warm (the simulator job costs only while it runs). That is a defensible cost for a submission, and far cheaper than a demo that cold-starts in front of judges.

10. Setup order

Everything here is done before code matters. The target is the Master Document's hard milestone: **a transaction visible in a deployed UI by 30 September.**

#	Task	Owner	By
1	Confirm with the organisers that planning inside the build window is fine (a confirmation, not a blocker)	Whoever registers	28 Sep
2	Register the team (window closes 11 October)	Whoever registers	11 Oct at the latest
3	Claim Google Cloud credits if the competition provides them	Backend owner	Week 1
4	Create GCP project `sentinel-ai-prod` in `asia-southeast1`; enable Cloud Run, Pub/Sub, Vertex AI, Artifact Registry, Cloud Build, Secret Manager	Backend owner	22 Sep
5	Request Vertex AI quota; verify current Gemini model IDs	AI owner	22 Sep
6	Create MongoDB Atlas cluster on GCP, same region; connection string into Secret Manager	Backend owner	23 Sep
7	Create the public GitHub repo, npm workspaces, first commit	Frontend owner	23 Sep
8	`packages/domain` — types, thresholds, demo constants	Frontend owner	23 Sep
9	Cloud Build pipeline deploying a hello-world container to Cloud Run	Backend owner	24 Sep
10	Simulator publishing to Pub/Sub	Data owner	24 Sep
11	`core` consuming, writing to Mongo	Backend owner	26 Sep
12	`stream` gateway + Angular shell receiving over WebSocket	Frontend owner	28 Sep
13	**Transaction visible in the deployed UI**	All	**30 Sep**

Why step 9 comes before any feature

A pipeline that deploys nothing useful, on day four, is worth more than a feature that runs only on a laptop. Once hello-world is live on Cloud Run through Cloud Build, every later step is an increment rather than an integration. Teams that leave deployment until week three lose days to it, always.

The go/no-go

If a transaction is not visible in a deployed UI on 30 September, scope comes out that day — the entity graph first, since it is the most self-contained module to drop.

11. Toolchain risks

Risk	Likelihood	Impact	Fallback
Submission material accidentally shipped in a language other than English	Medium	**Fatal**	Tamil material is internal only and never uploaded; one named person checks every submitted artefact against the English requirement before upload
Vertex AI quota denied or delayed on the project	Medium	Severe	Request in week one. If denied, the control path still works entirely without Gemini — that is the architecture's own fallback, and we say so
Gemini model ID retired before the finale	Medium	Severe	IDs in config, not code. Re-verify 5 Oct and again in the week before 4 Dec
MongoDB Atlas free tier insufficient (change streams, connections)	Medium	Moderate	Upgrade to M10; the cost is small and the decision is one dropdown
Cloud Run WebSocket connection drops mid-pitch	Medium	Severe	60-minute request timeout, client reconnects with backoff and resumes from a server-issued global stream cursor; server replays a snapshot
Antigravity or AI Studio workflow slows someone down in week one	Low	Moderate	Tooling is per-person and optional. VS Code locally is the default; nobody is required to learn a new environment during the build
Cold start on a scaled-to-zero service during the demo	Low	Severe	`min-instances=1` on all three services, set on day one and never lowered
Team loses a member in the final week	Medium	Severe	Every module has one named owner and a README section; no unmerged local work overnight

The two risks that are actually different in kind

Everything above except the first row is a technical problem with a technical fallback. **The language requirement is not.** English-only is checked at upload and nothing recovers from breaching it, so one named person owns that check across the proposal, README, code comments, deck and video captions. The eligibility worry raised in an earlier draft is now resolved: the build window opened on 7 September and all SENTINEL work began on 20 September, inside it.

The second is quieter: **adopting three new Google tools in week one because they look good in the submission.** Judges score the artefact, not the IDE. Use the tooling that makes each person fastest, name in the deck only what we genuinely used, and spend the saved days on the eight MVP modules.

12. Platform direction — what we deploy, what we only name

The v1.1 product direction (multi-tenant platform, four planes) adds no new Google Cloud services to the submission. It adds one route and three data shapes to services we already run. Everything else is named in the proposal as the production path and never shown as deployed.

Capability	Submission	Production service (named only)
Pre-transaction decision API	Route on `sentinel-core` (Cloud Run)	API Gateway in front, mTLS, private connectivity
Canonical event envelope	Same Pub/Sub topic, ordering key per account token	Pub/Sub dead-letter topic and replay
Dedupe	Unique index on `idempotencyKey` in Atlas	Same
Edge provenance	Fields on `entity_edges` in Atlas	Graph projection, e.g. Spanner Graph
Tenant lens	Client-side projection, synthetic tenants	Identity Platform multi-tenancy, per-tenant SAML/OIDC
Secrets and keys	Secret Manager	Plus Cloud KMS customer-managed keys

Fixed wording for platform claims

The same rule as COSMIC and SRF: one owner, one wording, used verbatim.

Say	Never say
Designed as a multi-tenant platform; demonstrated as a single deployment on synthetic data	Deployed to multiple banks
Pre-transaction decision API, called by our simulator	Integrated with bank payment systems
Cross-bank trace reconstructed from each institution's own events, with an explicit observed boundary	Full visibility of funds across Singapore banks
COSMIC-inspired cross-institutional risk intelligence	Connected to COSMIC
The prototype simulates three participating institutions using the same canonical event contract the production connector layer is designed to consume	We integrated three banks
SENTINEL applies the configured policy automatically; authorised bank operators manage verification, escalation and exceptions	An analyst can release any hold
Network-authorised device relationship	Shared device fingerprint across banks

The name stays **SENTINEL AI** across every artefact. Alternative names such as "ZEUS SHIELD" are not used in the submission; one name, one repo, one video.

Cost impact

None measurable. The decision route runs on the existing `min instances = 1` service; v1.2 adds three small Atlas collections (transaction_events, security_events, decisions) inside the same cluster tier. The cost table in section 9 stands.

Sources

- AI Builder Cup 2026 — FAQ
- AI Builder Cup 2026 — Themes and judging
- Google I/O 2026 developer highlights — Antigravity, Gemini API, AI Studio

- Firebase Studio sunset and project migration

Competition details verified 20 September 2026 and to be re-checked in the week of 28 September.